

GOOD DATABASE DESIGN

- 1) Requirements Analysis
- 2) Conceptual Database Design
- 3) Logical Database Design
- 4) Schema Refinement
- 5) Physical Database Design.
- 6) Application and Security Design.

Requirement Analysis

* The main aim is to understand the needs & requirements of Stakeholders.

Main steps

- 1) Identify stakeholders:- who will use your database system.
- 2) Gather requirements:- Conduct interviews or surveys to gather information
- 3) Define Functional & non-Functional requirements.

Conceptual Database Design

* The data specifications are converted into an ER-model.

* This is a high-level view of the entire database.

Logical Database Design Schema

- * Second stage of database design.
- * Converts conceptual into logical model.
- * This design is in the form of tables, columns & relationships.

Schema Refinement

- * Refinement process is called normalization.
- * This is a step by step process that decomposes a complex relation into a simple relations.
- * Different stages of normalization are known as "normal forms"

Key Points:-

- 1) Make sure that the semantics of the attributes is clear in the schema.
- * Entity & relationship attributes should be kept apart as much as possible.
- 2) Redundant information in Tuples & Update
↓
Anomalies,
reducing ~~redundant~~
- 3) Reducing the NULL values in tuples.
- 4) Disallowing the possibility of generating a false tuple.

Physical Database Design

- * This is the final stage of database design.
- * Defines storage structure, access methods, indexes, & other physical parameters.

Functional Dependency

- * In a given relation R , X & Y are attributes. ' Y ' is said to be functionally dependent on attribute ' X ' if each value of X determines exactly one value of Y . $X \rightarrow Y$.

Ex: $t_1[A] = t_2[A] \Rightarrow t_1[B] = t_2[B]$

A	B
1	4
1	5
3	7

- * Here t_1, t_2 values in A are equal so the values in the positions of t_1, t_2 in B should also be same this is called functional dependency.

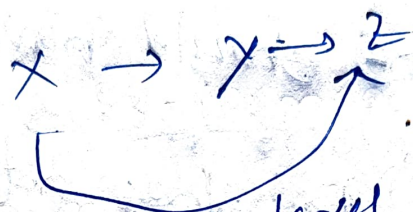
Ex:-

Reports (Student#, Course#, CourseName, IName, Room#, Marks, Grade)

- * Marks are functionally dependent on {Student#, Course#}

Types

- 1) Full Functional Dependency.
- 2) Partially Functional Dependency.
- 3) Transitive Dependency.



- 4) Trivial Dependency.

* In $X \rightarrow Y$ said to be trivial functional dependency if Y is a subset of X ($Y \subseteq X$).

Normalization

* It is an approach of decomposing tables to eliminate data redundancy & undesirable characteristics like insertion, update & deletion anomalies.

Properties

- Lossless join property.
- * Which do not generate false tuples.
- Dependency preservation property
- * Represents the functional dependency in some relation.
- * Normal forms are used to reduce redundancy in database tables.

First Normal Form (1NF)

- * Should contain only single valued attributes/columns.
- * Values in column should be of same domain.
- * All the columns should have unique names.
- * The order of data does not matter.
↓
stored.

Second Normal Form

- * IF a relation is in 1NF
- * No. partial dependency.

Third Normal Form

- * IF a relation is in 2NF
- * No transitive dependency
- * If non-prime attributes depends on other non-prime attributes rather than depending upon the prime attributes or primary key. This is transitive dependency.

Boyce - Codd Normal Form (BCNF)

- * This is possible if and only if all the determinants are candidate keys.
- * All BCNF relations are 3NF, but all 3NF are not BCNF.

Fourth Normal Form (4NF)

- * If the relation is in BCNF (3NF)
- * Relation should not contain multi valued dependency.

Fifth Normal Form (5NF) or Projection

Join Normal Form (PJ/NF)

- * If R is in 4NF.
- * Relation contains no JOIN dependencies or it can't be further non loss decomposed.
- * This normal form is not possible in real life.

Join dependency

- * R is relation schema $\{R_1, R_2, R_3, \dots, R_n\}$
be the decomposition of R ,
 $\pi_{R_1}(R) \bowtie \pi_{R_2}(R) \bowtie \dots \bowtie \pi_{R_n}(R) = R$
- * Order of join doesn't matter.

MULTI VALUED DEPENDENCY

* This occurs when two attributes in a table are independent of each other but both depend on a third attribute.

Decomposition

* Decomposition removes redundancy, anomalies & inconsistencies from a data base by dividing the table into multiple tables.

* This improves the overall efficiency & functionality of a database.

Types

→ Lossless Join Decomposition.

→ Lossy Join Decomposition.

Lossless Decomposition

* It is a process in database normalization where a relation is decomposed into multiple smaller relations (tables).

Ex:-

Table 1:- Emp

Emp-ID	Emp-Name	Emp-Age	Emp-Location
E001	Se Khar	25	Nizvid
E002	Manoj	26	Vijayawada
E003	Mari	24	Hyderabad

Table 2 :- < Dept >

Dept-ID	Emp-ID	Departments
Dpt 1	E001	Operations
Dpt 2	E002	HR
Dpt 3	E003	Finance

* Apply Natural Join for both the tables.

Emp-ID	Emp-Name	Emp-Age	Emp-Location	Dept-ID	Dept-Name
E001	SeKhar	25	Pune	Dpt 1	Ops
E002	Mang	26	Vij	Dpt 2	HR
E003	Hari	24	Hyd	Dpt 3	Finance

* There is no loss of information.

Lossy Decomposition

* This is a careless decomposition.

* In this the original relation cannot be perfectly reconstructed from the new decomposed relations.

Files

Files are used as a ~~survey~~ way to store and organize data in a structured manner for efficient retrieval & manipulation.

File:- A file is named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes & optical disks.

* In simple terms, storing and organizing the files in certain order is called file organization.

Types of File Organizations

1) Sequential File Organization

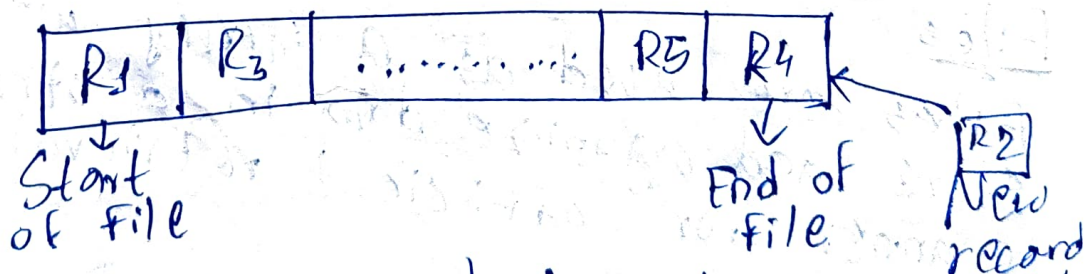
* In this the files are stored one after the other in sequential manner.

* There are two ways to implement this method.

- Pile File Method
- Sorted File method.

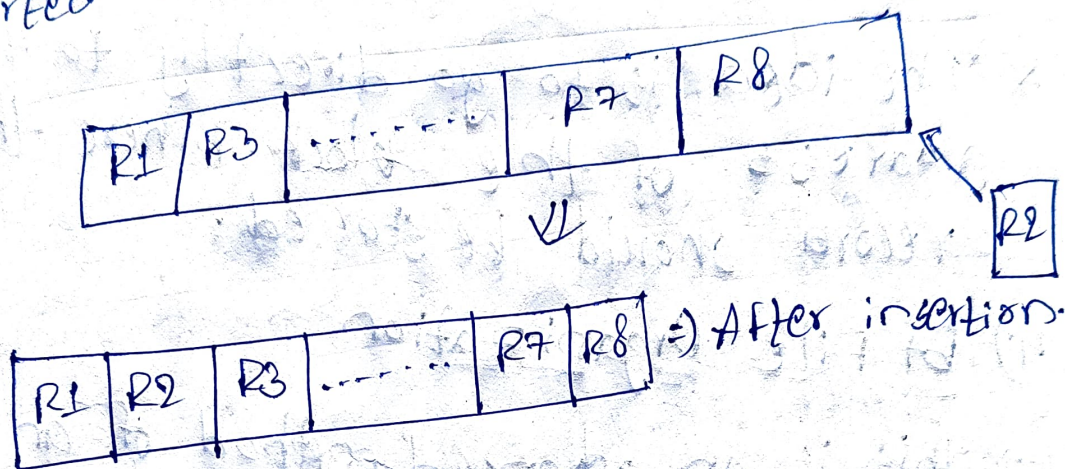
* In pile file method we store the records in a sequence one after other in the order in which they are inserted into the tables.

Ex:-



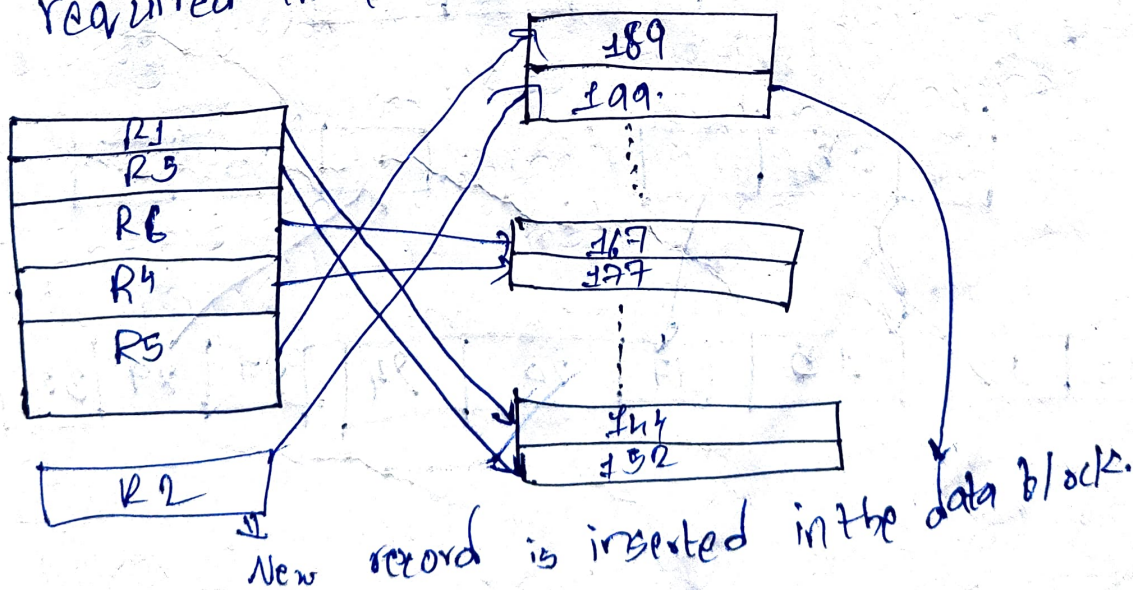
* When a new record has to be inserted it is done at the end of the file.

- * We cannot delete the record in sequential manner.
- * In sorted file method the records are inserted in a (ascending or descending) sorted manner.



2) Heap file organization

- * This works with data blocks. The records are inserted at the end of the file, into data blocks. No sorting or ordering is required in this method.



3) Hash File Organization

* It is a method of storing records in a database file where the physical address of the data block is directly calculated using a hash function.

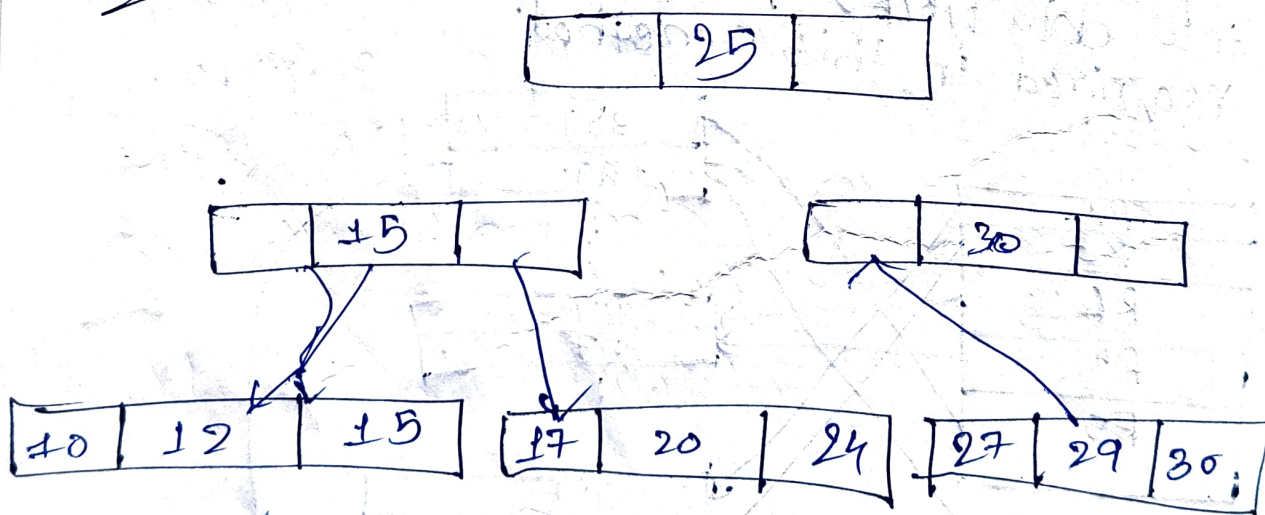
* The idea is to go directly to the address of the block where the record should be stored.

4) B+ File Organization

* This is an advanced method of an indexed sequential access method.

* This is a tree like structure.

Ex:-



Clustered file Organization

- * When two or more records are stored in the same file, it is known as clusters.
- * This organization is used when there is a frequent need for joining the tables with same condition.

PL/SQL (PL/SQL)

- * This is a block structured language. The PL/SQL programs are divided & written in logical blocks of code.

Syntax → DECLARE ⇒ (Declarations)
BEGIN, END ⇒ (Executable commands)

- * EXCEPTION ⇒ Exception Handling.

- * CREATE FUNCTION commands are used to write functions in PL/PG SQL.

Co-4 TRANSACTION

- * This is a set of logically related operations done on a database.
- * Transaction is performed to perform operations for accessing the contents of the database.

Six Operations

- 1) Read your acc balance
- 2) Deduct the amount from your balance
- 3) Write the remaining balance to your account.
- 4) Read your friend's account balance.
- 5) Add the amount to his account balance.
- 6) Write the new updated balance to his account.

Ex A's Account

* Open - Account (A)

Old-Balance = A-balance

New - Balance = Old-Balance - 800

A-balance = New-Balance

Close - Account (A)

* B's Account

Open - Account (B)

Old - Balance = B.balance

New - Balance = Old - Balance + 800

B.balance = New - Balance

~~Open~~
Close - Account (B)

→ T_i : read (A);

A := A - 800;
write (A);

read (B);

B := B + 800;

write (B).

Two main operations

→ read (A) :- which transfers the data

A from the database to a variable

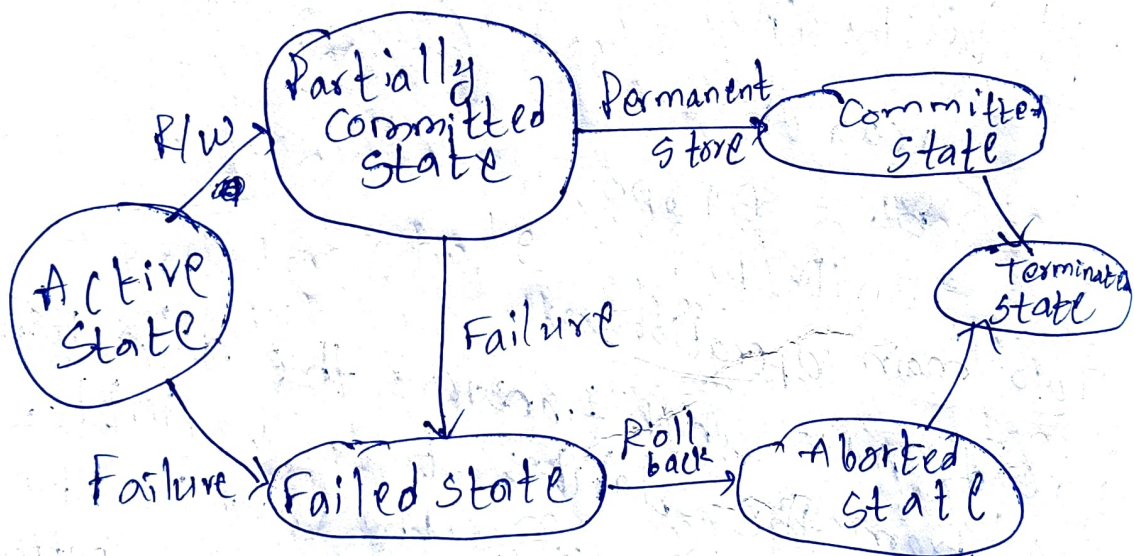
→ write (A) :- transfers the value in the
variable A in the main-memory buffer
of the transaction.

→ commit() :- used to save the
updates.

* multiple users can access & use the same database at one time, which is known as concurrent execution of the database.

Transaction States

- Active state
- Partially committed state
- Committed state
- Failed state
- Aborted state
- Terminated state.



Multiple transaction Processing Problems.

→ Lost update Problem

* This occurs due to the update of the same record by two different transactions at the same time!

* The first transaction updates a record & the second transaction updates the same record again, which nullifies the update of

the first transaction, Hence the update is lost of first transaction.

→ Ways to prevent lost updates

- * Increase transaction Isolation level.
- * Optimistic & Pessimistic Locking.

→ Unrepeatable Read Problem

Time	A	B
T1	READ(DT)	
T2		READ(DT)
T3	$DT = DT + 500$	
T4	WRITE(DT)	
T5		READ(DT)

* Here, first the data^(DT) is read in both A & B transactions as 1000. Then in 'A' the DT is updated to 1500 & then again 'B' reads DT transaction again & finds it as 1500, A + last 'B' finds two values for DT so this is the problem.

→ Phantom Read Problem

Time	A	B
T1	READ(DT)	
T2		READ(DT)
T3	DELETE(DT)	
T4		READ(DT)

* Here the data DT is read in both A & B transactions, then the value is deleted so, 'B' cannot find the value.

ACID properties

- Atomicity
- Consistency
- Isolation
- Durability

Atomicity

- * All change to data are performed as if they are a single operation. i.e; transaction occurs completely, or it does not occur at all.

Consistency

- * Data is in a consistent state when a transaction starts & when it ends.

Isolation

- * The intermediate state of a transaction is invisible to other transactions.
- * This property ensures that multiple transactions can occur simultaneously without causing any inconsistency.

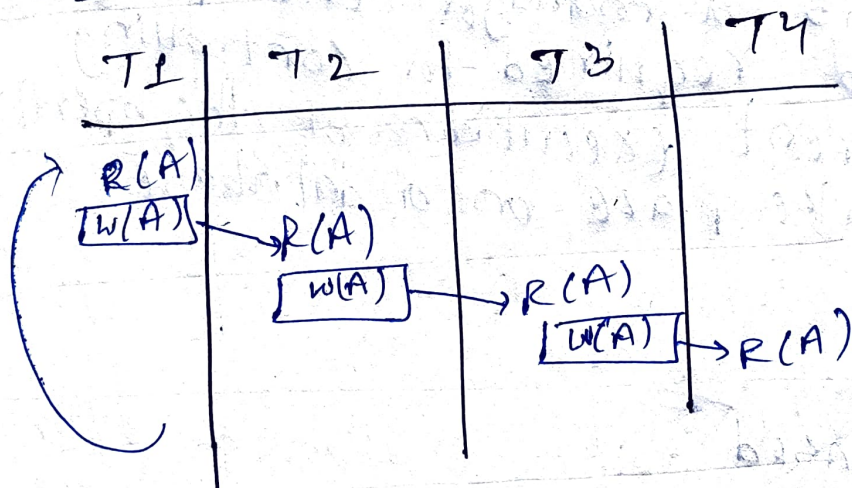
Durability

- * This ensures that all the changes made by a transaction after its successful execution are written successful to the disk.

TECHNIQUES USED TO ASSURE ACID PROPERTIES

- Atomicity $\Rightarrow$ Two-phase commit protocol (2PC)
- Consistency $\Rightarrow$ Conflict resolution.
- Isolation $\Rightarrow$ Locking.
- Durability $\Rightarrow$ Replication, Backup & recovery.

CASCADING ROLLBACK DUE TO DIRTY READ PROBLEM



* If transaction T1 writes uncommitted x that is read by T2, then T2 writes uncommitted x that is read by T3. In this case if T1 fails it must be rolled back then as T2, T3 are dependent on T1 they must also be rolled back.

* This is cascading roll back due to dirty read problem.

* Dirty read problem is prevented through cascadeless schedule.

* In cascadeless schedule, if a transaction is going to perform read operation on a value, it has to wait until the transaction who is performing write on that value commits.

Concurrency Control

* This is a management procedure that is required for controlling concurrent execution of the operations that take place on a database.

Protocols

- Lock based
- Time stamp
- Validation based

Lock based

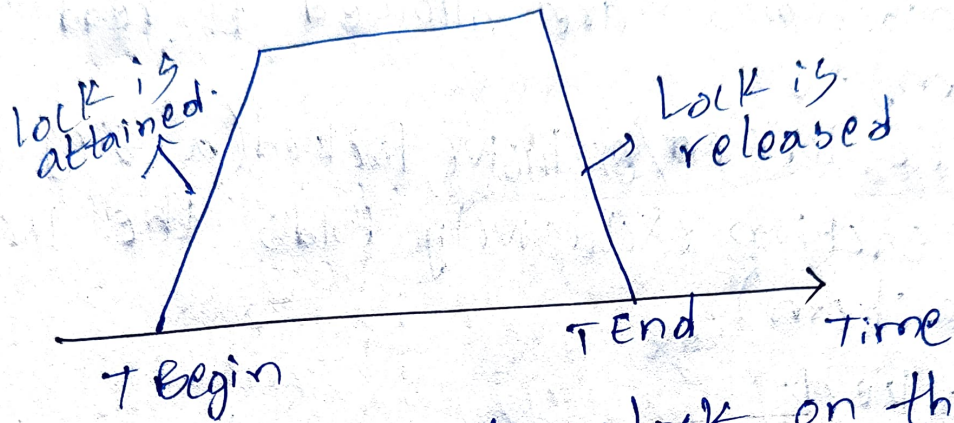
* Any transaction cannot read or write data until it acquires an appropriate lock on it.

Locking Mechanisms

- Shared lock
- Exclusive lock.

Types

- Simplistic lock protocol
- Pre-claiming lock protocol
- Two-phase locking (2PL)



* Growing Phase: - A new lock on the data item may be acquired by the transaction.

* Shrinking Phase: - Existing lock held by the transaction may be released.

* Lock Point: - The point at which growing phase ends.

Time stamps: - It is a unique identifier for each transaction, generated by the system.

Types of locks

→ Binary locks

- ↳ Locked (lock-item(x))
- ↳ Unlocked (unlock-item(x))

→ Shared / Exclusive Locks (Read / Write)

These allow concurrent access to the same item by several transactions.

* read locked / shared locked (other transactions are allowed to read the item).

* write locked / exclusive locked (a single transaction exclusively holds the lock on the item).

* Unlocked.

→ The above mentioned are the possible three states of shared / Exclusive locks.

Timestamp based

* Each transaction is issued a time-stamp when it enters the system. If an old transaction T_i has time-stamp $T_s(T_i)$, a new transaction T_j is assigned time-stamp $T_s(T_j)$ then

$$T_s(T_i) < T_s(T_j)$$

→ Write $- T_s(X)$

→ Read $- T_s(X)$.

Basic Timestamp Ordering

* This checks whether conflicting operations violate the time stamp ordering in two cases:

1) If read - $TS(X) > TS(T)$ (or) if write - $TS(X) > TS(T)$, then a younger transaction has already read the data item so abort & roll-back T & reject the operation.

2) If the above part does not exist, then execute write-item(X) of T & set write - $TS(X)$ to $TS(T)$.
Vice versa for write operations.

Strict Timestamp Ordering

* Checks the two conditions.

1) write-item(X) $\Rightarrow TS(T) > read-TS(X)$

* In this we should delay 'T' until other transaction completes.

2) read-item(X) $\Rightarrow TS(T) > write-TS(X)$

* In this also we have to delay 'T' until other transaction completes.

GRANULARITY

* This can affect the performance of concurrency control & recovery.

Coarse granularity: — larger the data item size, the lower the degree of concurrency.

fine granularity:— Smaller the data is, the more locks to be managed & stored.

SHADOW PAGING

- * When a transaction begins executing, the current page table is copied into a shadow page table.
- * Then shadow page table is saved.
- * Shadow page table is never modified during transaction execution.

Uses:-

- * To recover from a failure.
- * Committing a transaction.

Multiversion Timestamp Ordering

- * A data item 'x' has a number of versions ($x_0, x_1, x_2, \dots$); each version has its own read-TS & write-TS.
- * Requires more storage to maintain multiple versions.

→ read-TS(x_i)

→ write-TS(x_i)

• write-item(x) ⇒ $\text{write-TS}(x_i) \leq \text{TS}(T)$

Latest
version
before T
started.

→ This protocol to check next

$read_TS(x_i) > TS(T) \Rightarrow$ Here we should abort & roll back

$\cdot read_item(x) \Rightarrow write_TS(x_i) \leq TS(T)$
 $\Downarrow$
Read the version

Types of Failure

- $\rightarrow$ Transaction Failure $\Rightarrow$ incorrect input, deadlock, incorrect synchronization.
- $\rightarrow$ System failure $\Rightarrow$ Application error, O.S. fault.
- $\rightarrow$ Media failure $\Rightarrow$ Disk head crash, power disruption.

Log File

* Holds schedule of transactions.

Types of records (entries) in log file

- $\rightarrow$ [start - transaction, T]:
- $\rightarrow$ [write - item, T, x, old-value, new-value]:
- $\rightarrow$ [read - item, T, x]:
- $\rightarrow$ [end - transaction, T]:
- $\rightarrow$ [commit, T]:
- $\rightarrow$ [abort, T]:

Data Updates

- 1) Immediate Update:— Updates are applied to the database immediately after every write operation.
- 2) Deferred Update:— Updates are kept in main memory until the transaction commits.
- 3) Shadow Paging:— A duplicate copy of each page is made before updating. When commit occurs, the shadow becomes the new master.
- 4) Caching/ Buffering:— One or more disk pages that include data items to be updated are cached into main memory buffers & updated in memory.

→ UNDO = backward

→ REDO = forward

→ Steal = recovery may require UNDO

→ NO-steal = NO-UNDO

→ FORCE = NO-REDO

→ NO-FORCE = recovery may require REDO

Active Transaction List:- All active (uncommitted) transaction ids are entered in this list.

Committed Transaction List:- Transactions committed after the last check point are entered in this table.

* During recovery.

→ Transactions in committed list are redone.

→ Transactions in active list are ignored.

→ Redo is needed.

→ Transactions in active list are undone

→ Transactions in committed list are ignored.

→ Undo is needed.

Recovery Techniques

Logging:-

→ It is a recovery technique used in DBMS to record all modifications made to a database during a transaction.

Types of logs

→ Transaction log

→ System log

→ Archive log

→ Redo log.

Check pointing

- * Reduces the amount of work needed to recover the database after a failure.

Types

- 1) Time-based checkpoints.
- 2) Threshold based checkpoints.
- 3) Hybrid checkpoints.

Redo

- * It refers to the process of reapplying changes made to the database from the log during the recovery process.

Undo

- * It refers to the process of reversing changes made to the database during an incomplete transaction.

Recovery Manager

This ensures that the database remains consistent & durable, even in the event of a failure.

- Logging
 - Check pointing
 - Recovery
- } Components.

Transaction Management

- * Essential for maintaining the integrity of the database & ensuring that it remains consistent in the event of a failure.

Two types of transactions

- 1.) Atomic:- This is a single, indivisible unit of work that is executed as a single entity.
- 2.) Composite:- Composed of multiple atomic transactions that are executed as a single unit of work.

Shadow Paging

- * It contains two page tables:

- Current page table
- Shadow page table.

- * Offers durability & atomicity.

- * Two different environments,

- Single user
- multi-user.

RIES (Algorithm for Recovery & Isolation) Exploiting semantics

* This is an algorithm that is to deal with recovery for database that utilize STEAL & NO-FORCE Policies.

* This is a three phase algorithm:-

- Analysis phase
- REDO phase
- UNDO phase.

* It maintains two data structures & adds one more field to log record.

- Transaction table
- Dirty page table.

Indexing:-

* It is a data structure technique which allows you to quickly retrieve records from a database file.

Form of the index:-

< field value, pointer to record >

structure of an index:-

Search key	DATA Difference
------------	-----------------

Types of Indexing

- 1) Primary Indexing ↙ Dense
↘ Sparse } ordered file
- 2) Clustered Indexing
- 3) Secondary Indexing
- 4) Multi level Indexing } unordered file.

Primary Indexing

- * This has fixed length size with two fields.

Dense

- * A record is created for each search key valued in the database.
- * Searching is faster.

Sparse

- * This contains only anchor records.
- * To locate a record, we find the index record with the largest search key value $\leq$ search key value we are looking for.
- * Time Complexity = $\log 2N + 1$.

Clustered Indexing

- * In this we will group two or more columns together to get the unique values & create index out of them.

Secondary Indexing

- * It is an additional index created on non-primary key attributes of a database table to speed up data retrieval.
- * A primary index allows fast access based on the primary key.

Multi-level Indexes

- * Here a primary index is created to an index itself.
- * Original index file = first level index
- * The index created to the existing index is called the second-level index.

Dynamic Multilevel Indexes Using B-trees & B+ Trees

- * In B-tree & B+ Tree data structures, each node corresponds to a disk block.

B-Tree